From EDGAR Filings to a Structured Database using Claude Code

Video 3 in a series on Claude Code

PAUL GOLDSMITH-PINKHAM

MAR 29, 2026

36 7 2 Sh

Part 3 of a series on AI coding tools for empirical research, accompanying my Markus Academy video series.

Web scraping: Claude Code for Economists with Paul Goldsmith-Pink
Markus' Academy

Watch on

In the last post, we pulled structured data from a government API—homeownership rates by age, already in nice tabular form. But a lot of researcl data isn't like that. It's buried in PDFs, HTML filings, regulatory documents,

earnings call transcripts. Getting from “the information exists somewhere on internet” to “I have a queryable dataset” is often the hardest part of a project.

So we’re going to do exactly that: scrape SEC EDGAR, parse messy 10-K filings, extract specific sections, build a structured database, and run a simple analysi all through Claude Code. The whole pipeline takes about 30 minutes, and it produces an actual finding.

The question

Here’s what motivated this: the 2025 tariff escalation has been the dominant policy story in markets. If you follow trade-exposed firms at all, you’ve seen th stock price reactions, the earnings call commentary, the analyst reports. But h did firms change their *formal risk disclosures* in response? Item 1A of the 10-K the Risk Factors section – is where companies are legally required to describe material risks. It’s the section that lawyers agonize over. When a new risk emerges, it shows up here.

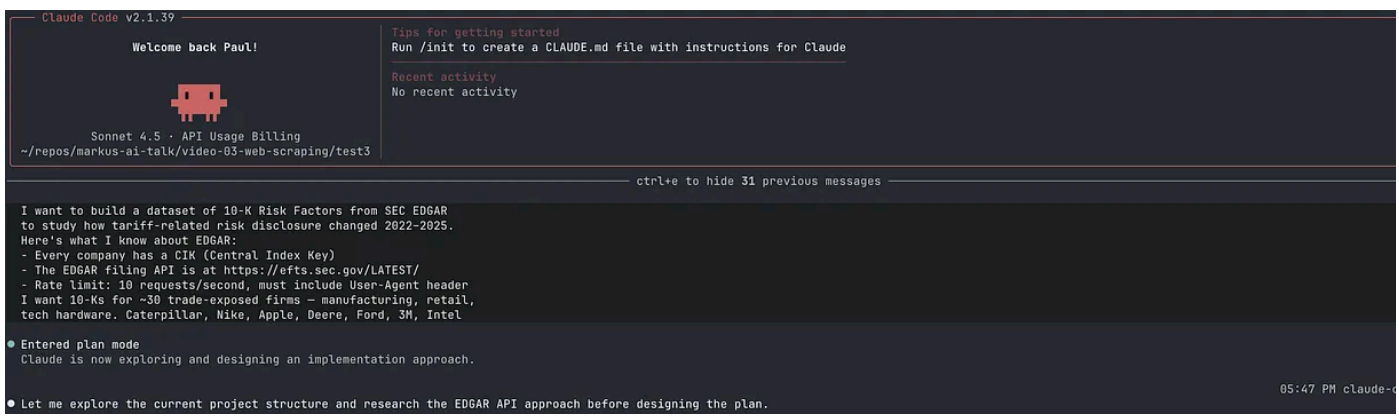
So the question is simple: *did firms increase their tariff-related language in 10 Risk Factors filings after the 2025 tariff escalation?* This connects to the broad literature on policy uncertainty measurement (think Baker, Bloom & Davis) an risk factor disclosure (e.g., Campbell et al., 2014). But today we’re just doing the descriptive work – building the dataset and seeing what’s there.

Scraping EDGAR

I start Claude Code in an empty directory. Unlike the last post where I was vag about the data source, here I know exactly where the data lives – SEC EDGAR and I can give Claude some useful context about how it works.

```
> ❯ I want to build a dataset of 10-K Risk Factors from SEC EDGAR
  to study how tariff-related risk disclosure changed 2022–2025.
  Here's what I know about EDGAR:
  - Every company has a CIK (Central Index Key)
  - The EDGAR filing API is at https://efts.sec.gov/LATEST/
  - Rate limit: 10 requests/second, must include User-Agent header
  I want 10-Ks for ~30 trade-exposed firms — manufacturing, retail,
  tech hardware. Caterpillar, Nike, Apple, Deere, Ford, 3M, Intel
```

If you remember from the first video, I talked about how well Claude does as a function of how clear and precise you are. The more precise I am, the less it ha to search randomly for things. Claude needs to know about rate limits and Use Agent headers *before* it writes the code, not after it gets a 403 error (of course, i still screwed up when doing this — I wasn't insistent enough).



Plan mode and interactive design

Something notable happened right away that didn't occur in the previous vide Claude entered *plan mode*. It looked at the prompt, recognized this was a substantial multi-step project, and said it wanted to design the approach befor writing any code. It researched the EDGAR API—fetching documentation, look at examples on GitHub, understanding the filing index structure—before proposing anything.

Then Claude did something I want to highlight: it asked me questions. Three c them:

1. **Database format**: "Do you want a SQLite database? A flat file? Both?" I told i wanted DuckDB—it's similar to SQLite but it's how I prefer to do things.
2. **Keyword approach**: "Do you want keyword search, full text extraction witl flagging, or LLM classification (which would cost more)?" I chose keyworc search—just look for words related to tariffs.
3. **User-Agent**: "What entity should we use for the User-Agent header? Do you want to fill it in later or provide it now?" I gave it my name and email right there.

Markus also had a good suggestion during the conversation: add "liberation da to the tariff keywords, since April 2, 2025 was the major tariff announcement. Claude immediately updated the keyword list to include "liberation day" and "liberation day tariff."

After this planning phase, Claude saved its plan to a file—just like the intentior compaction approach I described in post 1. The plan included the target compa list, the database schema, the keyword dictionary, and the pipeline architectur Then it cleared the context window but kept the plan on disk, so it could start fresh with the full context budget while retaining all the design decisions we'c made. This is exactly the pattern we talked about: research → write plan to file new session reads plan → execute.

Plan: SEC EDGAR 10-K Risk Factors Tariff Dataset

Context

Build a Python pipeline that downloads 10-K filings from SEC EDGAR for ~30 trade-exposed firms (2022–2025), extracts the Item 1A Risk Factors section, identifies tariff-related paragraphs via keyword matching, and stores everything in a DuckDB database for analysis.

Target Companies (~30)

Sector	Tickers
Manufacturing	CAT, DE, MMM, F, GM, BA, HON, EMR, ITW, CMI
Retail	NKE, WMT, TGT, HD, LULU, GPS, UAA, DLTR
Tech Hardware	AAPL, INTC, QCOM, CSCO, DELL, HPQ, AVGO, TXN, NVDA, AMD, MU, WDC

Architecture: Single script + DuckDB

One self-contained Python script (scrape_edgar.py) with clear phases:

Phase 1 — CIK Lookup

- Fetch https://www.sec.gov/files/company_tickers.json
- Map each target ticker → CIK number
- User-Agent: Paul Goldsmith-Pinkham paulgp@gmail.com

Building and running the pipeline

Claude wrote about 480 lines of Python to implement the pipeline. It has all th phases: CIK lookup, filing index queries, document downloading, HTML parsir with BeautifulSoup, and structured storage. The practical features matter—it caches downloaded HTML so you don't re-download on subsequent runs, it lo errors without stopping the batch, and it tracks every download attempt in metadata.

One useful feature: you can queue a follow-up question while Claude is runnin long process. While the downloader was chugging through filings, I typed my next question and it waited until the current task finished before answering.

```
> Run it for the full list. Let's see how it goes.
```

The script found CIKs for 29 out of 30 firms. The one failure: Gap Inc, which wa listed as ticker "GPS" in our list. Claude couldn't find a CIK for it, so I asked wha happened. It turned out Gap Inc. had rebranded and was traded under "GAP" ir some datasets, not "GPS." Claude looked up the correct CIK (39911), updated the

script to use the right ticker, and re-ran. Because the script had caching built i only needed to download four new files—the rest were already on disk. Took about 43 seconds.

I also demonstrated something useful during the download: I accidentally hit escape and interrupted Claude mid-process. But because of how the context window works, I could just type “please continue” and it picked up right where left off. (You can also hit Ctrl+O to toggle the “thinking” view and see Claude’s internal reasoning at any point.)

Extracting and structuring

Raw HTML files sitting in folders aren’t research data. You can’t query them, y can’t join them to Compustat, you can’t do anything without loading everythin into memory and parsing it every time. I need a *structured database*.

Claude wrote a parser that handles the formatting variations – and there are a lot. Some filings are wrapped in HTML with `<div>` tags and CSS styling. Some plain text with dots and dashes for formatting. The section headers sometime say “ITEM 1A” in all caps, sometimes “Item 1A.” with a period, sometimes they' inside an `<a>` tag. Claude’s parser tries multiple regex patterns and falls back gracefully.

The first pass extracted Item 1A from 112 out of 120 filings. I asked why eight w missing. Claude investigated and found that Honeywell and Intel had a formatting edge case: “Item 1A” and “Risk Factors” appeared on *separate lines* with a newline between them, but the regex required them on the same line. Claude fixed the regex to allow newlines and re-ran: 119 out of 120. The one remaining holdout was 3M, which had an unusual filing structure that would need manual attention.

This is the “95% fast, last 5% requires judgment” pattern. For a quick analysis, 1 out of 120 is fine. For a published paper, I’d go back and hand-check 3M’s filing Either way, the database is built.

Querying the database

Now the payoff. I have a structured DuckDB database and I can actually *ask questions* of the data.

```
> Which firms mentioned tariffs the most?
```

Claude wrote a query and came back with specific results. Deere (John Deere) leads with the most tariff mentions, increasing from three to four over the 202 2025 period. Manufacturing and tech hardware firms mention tariffs most frequently; retail firms mention them least. Walmart only started mentioning tariffs in 2025. AGO dropped from four mentions to zero.

The context snippets are useful too. In earlier filings, you see generic language “changes in trade policy could adversely affect our business.” In 2025, you see specific language referencing Section 301, specific tariff rates, and particular product categories. The specificity of the language tracks the specificity of the policy.

Extending to 2010

I then asked Claude to download the 2010 10-Ks for the same set of firms and r the same tariff keyword analysis. The comparison was interesting: even in 20 18 out of the roughly 30 companies mentioned tariffs. But the *intensity* was different—25 total tariff paragraphs across 18 firms in 2010, versus 26 to 35 per year in the 2022-2025 period across roughly the same number of firms. The vocabulary had also shifted: in 2010, the terms were narrower—“tariff,” “trade

restrictions," "anti-dumping," "countervailing duties." By 2025, the language ha expanded to include "trade tension," "trade dispute," "trade war," "liberation day The breadth of the risk disclosure, not just its frequency, had changed.

This is a finding. Not a causal result – you'd need more structure for that – bu clean descriptive fact that would motivate a deeper analysis. If you were interested in the cross-section—which types of firms saw the biggest increase you could extend this to all 500 S&P firms without changing the pipeline. The cost of expanding the sample is low. We did the whole thing in about 30 minu

```
● Done. 28 new 2010 filings downloaded (AVGO and DELL didn't have 2010 10-Ks), 147/148 extracted. Now let me show the comparison.

● Bash(python3 -c "
      import duckdb…)
  └  ═══ Tariff paragraphs: 2010 vs 2022-2025 by company ═══
       Ticker  Sector            2010  2022  2023  2024  2025  Total
       ------------------------------------------------------------------
     … +48 lines (ctrl+o to expand)

● Here's the comparison:

  2010 vs 2022-2025 aggregate:
  - 2010: 25 tariff paragraphs across 18 companies
  - 2022-2025: 26-35 paragraphs per year across 16-22 companies

  Key observations:
  - Tariff risk language was already widespread in 2010 (18 of 28 companies with 2010 filings mentioned it), but the intensity has increased — more paragraphs per company in recent years
  - DE (Deere) has gone from 2 paragraphs in 2010 to 4 in 2024-2025
  - AMD was already high in 2010 (3 paragraphs) and remains so
  - LULU had 3 tariff paragraphs in 2010 but dropped to 0-1 recently
  - NKE and ITW mentioned tariffs in 2010 but stopped in 2022-2025
  - HON and UAA jumped to 3 paragraphs in 2025 — new concern
  - WMT mentioned tariffs in 2010, went silent 2022-2024, then returned in 2025

  Keyword evolution:
  - 2010 keywords were narrower: mainly "tariff", "trade restriction", "anti-dumping", "countervailing" (traditional trade remedy terms)
  - 2022-2025 added more geopolitical language: "export control", "trade tension", "trade dispute", "trade war", "reciprocal tariff"
```

The final pipeline

```
project/
├── data/
│   ├── firm_list.csv                          (input: tickers)
│   ├── filings/
│   │   ├── AAPL/
│   │   │   ├── AAPL_2022_10K.html
│   │   │   ├── AAPL_2023_10K.html
│   │   │   └── ...
│   │   ├── CAT/
│   │   └── ...
│   ├── filings.db                             (DuckDB database)
│   └── metadata.csv                           (download tracking)
```

```
├── figures/
│   └── tariff_disclosure.pdf
├── download_filings.py
├── extract_and_structure.py
├── analyze.py
├── requirements.txt
├── Justfile
└── README.md
```

The key thing is the database. `filings.db` is the reusable research asset – not the raw HTML files, not the text extractions. Once you have a structured database, you can:

- Add new keywords without re-parsing all the filings
- Join to other datasets (stock returns around filing dates, firm characteristi from Compustat)
- Run SQL queries directly instead of writing Python parsing code every tim
- Hand the database to a coauthor who doesn't need to re-run the scraping pipeline

This is the pattern for any text-as-data project: scrape → parse → structure → analyze. The raw files are an intermediate product. The database is the deliverable.

A few things I've learned

1. **Front-load context for scraping tasks.** Tell Claude about rate limits, authentication, API structure, and data format *before* it writes the code. W EDGAR specifically, mentioning the User-Agent requirement and the 10 req/sec limit upfront avoids a whole round of debugging.

2. **Let Claude ask you questions.** This was different from the earlier videos. Rather than specifying every detail upfront, I gave Claude the high-level ta and it asked me the design decisions: database format, keyword approach, user-agent identity. These are research decisions—I should be making the but I don't need to anticipate every one in the initial prompt.
3. **Build the database schema yourself.** Or at least approve it. What tables do you need? What keywords matter? What metadata should you track? Clau proposed a schema, but the choices—like tracking keyword counts with context snippets for spot-checking—reflect domain knowledge about wha I'd need downstream. That's the researcher's job.
4. **Resume capability is not optional.** When you're downloading 100+ files ov the network, things *will* fail. Timeouts, rate limit errors, missing filings, server glitches. The Gap Inc. ticker issue is a good example: because the script cached everything, fixing the ticker and re-running only required downloading four new files instead of re-doing all 120.
5. **Quality reports save you later.** The extraction quality report – which filing parsed successfully, word count distributions, flagged suspiciously short extractions – is how we caught the Honeywell/Intel regex issue and the 3 edge case. Without it, you'd discover months later that part of your sample has garbage data.
6. **DuckDB (or SQLite) is underrated for research.** Most economists I know work with CSVs and dataframes. But for text-as-data projects, a SQL-queryable database is a much better fit. Zero configuration (it's a single file queryable with SQL (which Claude writes fluently), and it handles the relational structure naturally – filings have many keywords, firms have many filings. You don't need Postgres or a server. Just a `.db` file. I prefer DuckDB, but SQLite works too.
7. **The finding matters.** It's tempting to treat scraping as a pure infrastructure task – "I built a pipeline, done." But the whole point is to learn something.

Even a simple descriptive analysis (tariff keywords increased, the vocabul shifted from narrow trade terms to broader policy-risk language, and Walmart didn't mention tariffs until 2025) gives you confidence that the pipeline is working and the data is real.

What's next

In the next post, we tackle what happens when your dataset gets big – too big RAM, too slow for a single machine. We'll cover DuckDB, parallelization, and running jobs on remote clusters, all orchestrated through Claude Code.

Subscribe to A Causal Affair

By Paul Goldsmith-Pinkham · Launched 3 years ago

Causal Inference, Coding and Consumer Finance

Type your email... Subscribe

By subscribing, you agree Substack's Terms of Use, and acknowledge its Information Collection Notice and Privacy Policy.

---

36 Likes · 2 Restacks

Discussion about this post

Comments Restacks

Write a comment...

Adam 4d

Liked by Paul Goldsmith-Pinkham

Really cool. Take a look at this plugin — it's been a gamechanger for me in projects such as this: https://github.com/EveryInc/compound-engineering-plugin

LIKE (2) REPLY

Florian Hollenbach 4d

Liked by Paul Goldsmith-Pinkham

I never really worked with DuckDB until I started using Claude Code, but I also noticed that Clauc really likes to use DuckDB and it has been super useful to learn.

LIKE (2) REPLY

5 more comments...

© 2026 Paul Goldsmith-Pinkham · Privacy · Terms · Collection notice
Substack is the home for great culture